Davante

Desarrollo de Aplicaciones Web
Desarrollo Web Entornos Cliente

Tema 3.
El lenguaje de programación de clientes

Diego Rodero Pulido

Tarea 3. Variables, tipos y conversiones

Tema 3. El lenguaje de programación de clientes · Tarea individual · Calificación: 5 puntos de rúbrica + hasta 5 puntos en la defensa en clase.

Objetivo

Practicar lo básico del lenguaje —variables, tipos y conversiones— y acostumbrarte a predecir lo que hará JavaScript antes de ejecutarlo. Al terminar la tarea debes ser capaz de:

- Declarar variables con `const` o con `let`, según vayan a cambiar o no, y comprobar su tipo con `typeof`.
- Convertir entre tipos con `Number()`, `String()` y `Boolean()`, y anticipar cuándo el resultado será `NaN`.
- Explicar la coerción de tipos y por qué en este curso se compara siempre con `===`.
- Construir textos con plantillas de cadena y presentar tu trabajo en una página maquetada con Bootstrap.

Qué vas a construir

En la carpeta `tema03` de tu repositorio de DWEC vas a completar una página, «Variables, tipos y conversiones», que funciona como un cuaderno de laboratorio: para cada ejercicio enseña el código, lo que esperabas que pasara y lo que pasó de verdad. El código se ejecuta con un botón y muestra sus resultados en la consola (y con `alert()` cuando se indique).

Para que te centres en programar, **no empiezas de cero: te doy una plantilla.**

- Está en la carpeta `tema03_plantilla` del GitHub de la asignatura: https://github.com/DRodero/DWEC_2627. Cópiala en tu repositorio y cámbiale el nombre a `tema03`. Trae esto:

```
tema03_plantilla/
├── index.html      navbar, cabecera y la card del ejercicio 1 (modelo)
├── js/
│   └── app.js      una función por ejercicio, con un ejemplo y TODO
├── capturas/       imágenes de relleno con el nombre de cada captura
└── README.md       el guion del README, con sus apartados
```

- Todo lo que va entre **[corchetes]** es un hueco: sustitúyelo por lo tuyo. Y cuando completes un `TODO`, borra el comentario.

Requisitos de la página:

- Pon **tu nombre y apellidos** en la `navbar`, en el `<title>` y en el pie de la página.
- Una `card` por ejercicio. La del ejercicio 1 ya está hecha y es el modelo: para los ejercicios 2, 3 y 4, cópiala entera (de `<section>` a `</section>`) y cambia el título, el código, la tabla «Espero / Sale» y el número del botón.
- El botón «Ejecutar» lleva `onclick="ejercicio1()"`: al pulsarlo, se ejecuta esa función de `js/app.js`. Cómo funciona lo veremos más adelante, con los eventos; de momento, cópialo y cambia solo el número.
- Todo el JavaScript va en `js/app.js`, dentro de la función de cada ejercicio, donde pone `TODO`. Cada función trae un ejemplo que te sirve de modelo.
- **Solo `alert()` y consola.** El JavaScript no escribe nada dentro de la página: las columnas «Espero» y «Sale» las rellenas tú, a mano, en el HTML. Escribir en la página desde JavaScript lo veremos más adelante.
- `let` y `const`, nunca `var`; comillas rectas; y las comparaciones, con `===`.

Si el código que copias dentro de `<pre><code>` lleva los signos `<` o `>`, escríbelos como `<` y `>`, porque si no el navegador los tomará por etiquetas.

Ejercicio 1. Variables y typeof

1. Declara al menos seis variables, una de cada tipo: `number`, `string`, `boolean`, `null`, `undefined` y `bigint` (un entero terminado en `n`, como `10n`).
2. Usa `const` para las que no cambian y `let` para al menos una a la que des valor más tarde.
3. Muestra en la consola el valor y el `typeof` de cada una.
4. En la página, una tabla con cada variable, su valor, el `typeof` que esperas y el que sale. Explica en una línea qué pasa con `null`.

Ejercicio 2. Conversiones explícitas

1. Haz al menos ocho conversiones con `Number()`, `String()` y `Boolean()`. Tienen que estar, como mínimo, estas: `String(123)`, `Number("123")`, `Number("12abc")`, `Number("")`, `Number(true)`, `Boolean(0)`, `Boolean("texto")` y `Boolean("")`.
2. **Antes de ejecutar**, escribe en un comentario de cada línea el resultado que esperas.
3. Muestra en la consola el resultado de cada conversión y su `typeof`.

4. En la página, una tabla «Conversión / Espero / Sale». Marca con un `badge` de Bootstrap las filas en las que te equivocaste: equivocarse no resta, esconderlo sí.

Ejercicio 3. Coerción y comparaciones

1. Comprueba en la consola seis expresiones que mezclen tipos. Puedes partir de las de la presentación (`"5" - 2`, `"5" + 2`…), pero al menos dos tienen que ser tuyas.
2. Compara con `==` y con `===` tres parejas de valores: `5` y `"5"`, `0` y `false`, y `null` y `undefined`.
3. En la página, una tabla «Expresión / Espero / Sale» y una frase tuya que explique cuándo manda la cadena y cuándo manda el número.

Ejercicio 4. Tu ficha con plantillas de cadena

1. Declara con `const` tus datos: nombre, ciclo, curso y una afición. Añade una variable con `let` que cambie de valor, por ejemplo las horas que has estudiado esta semana, y súmale algo con `+=`.
2. Construye con una plantilla de cadena (backticks y `${ }`) un mensaje que junte todos los datos, y muéstralo con `alert()` y en la consola.
3. Escribe el mismo mensaje concatenando con `+` y muéstralo también en la consola. Compáralos con `===`: tiene que salir `true`.
4. En la consola del navegador (no en `app.js`), intenta dar otro valor a una `const`. Copia el mensaje de error en la página, dentro de un `alert` de Bootstrap, y explica qué significa.

Evidencias y reflexión (README.md)

En el `README.md` de la carpeta explicas lo que has hecho y lo demuestras; la plantilla trae el guion con los apartados. Las capturas van en la carpeta `capturas/`, tienen que estar hechas en tu equipo y cada una lleva debajo una o dos líneas que expliquen qué se ve.

1. Cinco capturas propias, que sustituyen a las imágenes de relleno de `capturas/` con el mismo nombre: la página entera, con tu nombre visible, y la consola al ejecutar cada uno de los cuatro ejercicios (la del ejercicio 4, con el error de la const).
2. Una reflexión de 5 a 8 líneas: ¿qué conversiones te resultaron más intuitivas y cuáles te sorprendieron? Pon ejemplos concretos de tus tablas.
3. Fuentes consultadas, con su enlace, y el apartado «Uso de IA» si corresponde.

Entrega

- Todo va en la carpeta `tema03` de tu repositorio público de DWEC, con esta estructura:

```
tema03/
├── index.html     la página con los cuatro ejercicios
├── js/
│   └── app.js     una función por ejercicio
├── capturas/      la página y la consola de cada ejercicio
└── README.md      capturas, reflexión y fuentes
```

- Sube el trabajo en varios commits a medida que avances (tres como mínimo), con mensajes que digan qué cambia: «Ejercicio 1: variables y typeof», «Tabla de conversiones»...
- En la tarea del campus virtual entregas solo la URL de la carpeta: `https://github.com/tu-usuario/tu-repositorio/tree/main/tema03`
- Extensión orientativa del README: de 200 a 400 palabras, más las capturas.
- Se corrige el último commit anterior a la fecha límite publicada en la propia tarea del campus; lo que subas después no cuenta. No se admiten entregas por otros medios.

Criterios de evaluación (rúbrica)

La tarea se califica sobre 5 puntos con la rúbrica siguiente. Los otros 5 puntos, hasta el 10, se obtienen en la defensa en clase.

Criterio (puntuación máxima)	Insuficiente	Adecuado	Excelente
1. Página a partir de la plantilla (0,5 p)	0. No parte de la plantilla, no se ve tu nombre, faltan cards o quedan huecos [entre corchetes].	0,25. Tu nombre en la navbar y una card por ejercicio con su código, su tabla y su botón.	0,5. Las cuatro cards completas y ordenadas, código legible en los bloques pre y fallos de predicción marcados con badges.
2. Variables y typeof (0,75 p)	0 – 0,25. Menos de seis tipos, uso de var o el typeof no aparece en la consola.	0,5. Seis tipos declarados con let y const, su typeof en la consola y la tabla completa.	0,75. Elige bien entre const y let, da valor más tarde a una let y explica con sus palabras el caso de null.
3. Conversiones y coerción (1,5 p)	0 – 0,5. Faltan conversiones obligatorias o no hay predicción antes de ejecutar.	0,75 – 1. Ocho conversiones y seis expresiones con su predicción y su resultado; comparaciones con == y ===.	1,25 – 1,5. Explica con criterio propio cuándo manda la cadena y cuándo el número, y por qué === es más seguro.
4. Plantillas de cadena y error de const (0,75 p)	0 – 0,25. No hay plantilla de cadena o el mensaje no sale en alert() ni en la consola.	0,5. Ficha con backticks en alert() y en la consola, y la misma ficha con +.	0,75. Además, copia y explica con sus palabras el error de dar otro valor a una const.
5. Evidencias, reflexión y GitHub (0,5 p)	0. Sin capturas propias, reflexión genérica o todo el trabajo en un único commit.	0,25. Cinco capturas propias comentadas, reflexión de 5 a 8 líneas y varios commits.	0,5. Capturas con tu nombre visible, reflexión con ejemplos concretos y commits que muestran el avance.
6. Calidad del trabajo entregado (1 p)	0 – 0,25. Código desordenado o sin sangrar, comillas curvas, var, restos de la plantilla (huecos o TODO sin quitar) o README descuidado y con faltas.	0,5 – 0,75. Código ordenado y legible, README completo y pocas erratas.	1. Código limpio y comentado, nombres claros en camelCase y README cuidado, sin faltas.

Defensa en clase (hasta +5)

La defensa vale **la mitad de la nota**: es donde demuestras que sabes hacer lo que has entregado. No se trata de repetirlo: te pediré uno o dos cambios pequeños, los harás en el momento explicando lo que haces y te preguntaré por tu código.

- **Cuándo:** en la sesión de clase que indique después del cierre del plazo. Unos cinco minutos por persona.

- **Requisito:** solo puedes defender la tarea si la has entregado en plazo. Si no la defiendes, tu nota es la de la rúbrica (máximo 5).
- **Cómo:** abres tu página desde la versión entregada, la enseñas funcionando con la consola abierta, haces en tu propio código el cambio que te pida y respondes a lo que te pregunte sobre él.
- **Sin IA:** durante la defensa desactiva Copilot o cualquier autocompletado con IA. Puedes consultar los apuntes y la documentación de MDN.

Ejemplos de lo que te puedo pedir

- Añadir una variable nueva y enseñar su `typeof` en la consola.
- Predecir el resultado de una conversión o de una expresión que te diga, y comprobarlo.
- Cambiar un `==` por `===`, o al revés, y explicar por qué cambia el resultado.
- Pasar a plantilla de cadena un mensaje que esté concatenado con `+`.
- Explicar por qué `typeof null` devuelve `"object"` o por qué `Number("12abc")` da `NaN`.
- Encontrar y corregir, con ayuda de la consola, un error que yo introduzca en tu código.

Cómo se puntúa

Puntos	Qué demuestras
5	Haces el cambio sin ayuda y con soltura, explicas con seguridad tu código y lo que aparece en la consola, y respondes bien a lo que te pregunto.
4	Haces el cambio sin ayuda, con alguna duda menor, y explicas bien lo esencial.
3	Haces el cambio con una pista pequeña y explicas correctamente la mayor parte de tu código.
2	Necesitas varias pistas para hacer el cambio o tu explicación tiene lagunas importantes.
1	No consigues hacer el cambio sin mucha ayuda y solo sabes explicar partes sueltas de tu código.
0	No consigues hacer el cambio ni explicar tu propio código, o no te presentas.

Se pueden dar valores intermedios de medio punto.

Nota final = rúbrica (máximo 5) + defensa en clase (máximo 5).

Uso de la inteligencia artificial

Puedes apoyarte en herramientas de IA para orientarte, entender un error o revisar la redacción, pero el código, las predicciones y las explicaciones tienen que ser tuyos. Si las utilizas, indícalo al

final del README en un apartado «Uso de IA»: qué herramienta, para qué y qué hiciste después con su respuesta. Declararlo no resta nota.

Advertencia: si se detecta que la tarea se ha generado con IA sin trabajo propio, la calificación será un 0 absoluto.

Son indicios: la falta de evidencias propias, unas predicciones que aciertan siempre y no se equivocan nunca, el código con técnicas que no hemos visto y que no sabes explicar, los textos genéricos que no responden a lo que se pide, subir todo el trabajo de golpe en un único commit y no saber explicar lo entregado. Una captura tomada de internet, prestada o generada cuenta como el mismo caso.

La defensa en clase sirve precisamente para comprobarlo: si en ella no eres capaz de explicar nada de lo que has entregado, no solo no sumas esos 5 puntos, sino que se considera un indicio claro de que la tarea no es tuya.